- Where does enforcement take place? Inside a single platform like AWS, across microservices, or in a distributed environment?
- How portable should policies be? Will we need them to survive a platform change?
- Who will maintain them? Will policy authors be developers, system administrators, or domain experts who lack advanced programming skills?
- What's our tolerance for complexity? Some frameworks offer fine-grained control but come with steeper learning curves.

Ultimately, the best choice depends on the mix of authorization models you need, your performance requirements, and the developer experience your team can provide. By using the PARC model and the strength map together, you can move from abstract feature lists to a clear, practical choice.

8.6 Summary

- Over the years, a variety of policy languages and frameworks have been developed to handle different needs.
- XACML pioneered standardized ABAC but saw limited adoption due to verbosity and complexity; ALFA emerged to make it more approachable.
- OPA/Rego brings flexible, general-purpose policy-as-code to cloud-native systems, often used for Kubernetes admission control and microservice authorization.
- OpenFGA implements a relationship-based model inspired by Zanzibar, ideal for sharing and delegation scenarios.
- AWS IAM is deeply integrated with AWS services, making it the default choice for in-cloud authorization, though it's not portable beyond AWS.
- Applying PARC reveals how each system emphasizes different aspects of authorization, providing guidance on which might fit best in a particular environment.
- Selecting an authorization framework is partly dictated by factors beyond your control, such as the cloud vendor you use or the software